Relatório 8 — Verificador de senha com Flutter

Nome: [PREENCHER] | RA: [PREENCHER]
Polo: [PREENCHER] | Local da aula prática: [PREENCHER]
Data acadêmica: [PREENCHER]. Desenvolvimento remoto no Codex Cloud; sem declaração de participação presencial.

Objetivo e fundamentos

Criar uma interface Flutter que analise localmente uma senha e atualize sua classificação. Validação de dados aplica regras explícitas às entradas. Uma senha longa e única tende a resistir melhor a tentativas de adivinhação, mas uma regra de tamanho e classes de caracteres não mede sozinha a segurança efetiva.

Flutter organiza a tela em widgets. MaterialApp define o tema; Scaffold estrutura a página; TextField oculta o texto e usa onChanged; setState atualiza um Text com rótulo e cor. O resultado é comunicado por texto, além da cor, e Semantics com liveRegion favorece a indicação da mudança a tecnologias assistivas.

Critérios adotados

O roteiro não define limites numéricos. Nesta implementação, Forte exige pelo menos 12 caracteres e quatro classes; Média exige pelo menos 8 caracteres e duas classes, desde que não seja Forte; os demais casos são Fraca. As classes são minúsculas ASCII, maiúsculas ASCII, dígitos e símbolos diferentes de espaço. A contagem usa runes (pontos de código Unicode), não unidades UTF-16. Campo vazio solicita digitação, sem classificação.

A heurística é didática: não detecta padrões previsíveis, vazamentos, reutilização ou entropia; letras acentuadas entram na classe de símbolos desta regra simplificada. Não há armazenamento nem logs do texto digitado. Usar apenas entradas fictícias nos testes.

Lógica e código integral

```
if (tamanho >= 12 && classes == 4) return 'Senha Forte';
if (tamanho >= 8 && classes >= 2) return 'Senha Média';
return 'Senha Fraca';
```

Código integral: flutter/relatorio08-senhas/lib/main.dart. A função classificarSenha é independente da tela. O callback onChanged calcula o resultado e setState atualiza a interface; o campo vazio restaura a instrução inicial.

Testes e resultados reais

Flutter 3.35.7 e Dart 3.9.2 instalados com SHA-256 oficial. flutter pub get, flutter analyze, flutter test e flutter build web aprovados. A suíte executou 2 teste(s) de widget/lógica. O aplicativo compilado também foi servido localmente e testado em Chromium: 4 verificações de interface aprovadas. Logs e registros: testes/flutter-validacao-08.json e flutter-funcional-08.json.

Para reproduzir no SDK instalado: executar flutter pub get, flutter analyze e flutter test. Compilar para web e abrir no navegador. Usar três entradas exclusivamente fictícias que atendam às regras e registrar cada rótulo. Verificar também retorno à instrução inicial ao apagar o campo. Não registrar senhas reais em logs, capturas ou relatórios.

Conclusão e aplicação prática

A implementação explicita as regras para permitir testes reproduzíveis. Em sistemas reais, um indicador deve orientar sem prometer segurança absoluta. São necessários senhas longas e exclusivas, proteção de transporte, armazenamento no servidor com algoritmo apropriado de derivação de senha e autenticação multifator. A execução no navegador confirmou os três níveis e o retorno à instrução inicial ao limpar o campo.

Evidências autênticas e resultados

Aplicativo Flutter real compilado e executado em Chromium. 4 verificações de interface aprovadas; apenas dados fictícios, sem armazenamento ou logs de senhas. Os PNG originais permanecem em evidencias/relatorio08.



Figura 1 — Classificação Senha Fraca. Fonte: execução própria em Chromium.



Figura 2 — Classificação Senha Média. Fonte: execução própria em Chromium.



Figura 3 — Classificação Senha Forte. Fonte: execução própria em Chromium.

Requisitos técnicos e capturas concluídos. Identificação acadêmica permanece a preencher pelo estudante.

Referências

UNIVERSIDADE PAULISTA. Roteiros: Desenvolvimento Mobile. [S. l.]: UNIP, [s. d.]. Roteiro 8. Material didático fornecido.

UNIVERSIDADE PAULISTA. Desenvolvimento Mobile: relatório de aulas práticas. [S. l.]: UNIP, [s. d.]. Relatório 8. Material didático fornecido.

FLUTTER. Framework library: widgets e ciclo de vida. [S. l.], [s. d.]. Disponível em: https://raw.githubusercontent.com/flutter/flutter/stable/packages/flutter/lib/src/widgets/framework.dart. Acesso em: 31 out. 2026.

OWASP FOUNDATION. Authentication cheat sheet. [S. l.], [s. d.]. Disponível em: https://raw.githubusercontent.com/OWASP/CheatSheetSeries/master/cheatsheets/Authentication_Cheat_Sheet.md. Acesso em: 31 out. 2026.